



Universidad Autónoma de Baja California

**Facultad de Ingeniería, Arquitectura y
Diseño**

Ingeniería en Software y Tecnologías Emergentes



Arquitectura de la Información

Asignatura:

Tecnologías Emergentes para el Desarrollo de Soluciones

Docente:

Antonio de Jesus Garcia Chavez

Equipo:

Alcantar Martinez Emir Alexander | 379380

Meza Espinoza Kevin Andre | 379140

Montano Valencia Mike Armando | 379746

Moreno Calderon Troy Leonardo | 379169

Perez Aguirre Mextli Citlali | 379229

Jaime Mascareño Carlos Alberto | 377175

Zazueta Medrano Aidan | 379479

Fecha:

Ensenada, B.C a 22 de Septiembre del 2026

Fase 0 — Cimientos: Design Space Exploration

Insumo: Entrevista a Jonatan Crespo Ragland (profesor de asignatura, Facultad de Ingeniería) — docente que usa el QR de forma ocasional (perfil mainstream), en contraste con el experto ya entrevistado (Alfonso Ibarra Collins, administrador).

Necesidades identificadas

Jonatan nota problemas con cierta frecuencia (al menos una vez por semestre) pero reporta poco — hay fricción entre notar algo y decidir reportarlo.

Describe el sistema actual como "no es de fácil acceso" y pide una interfaz intuitiva — necesita simplicidad de uso.

Duda si un QR pegado en un salón es legítimo, porque "cualquiera puede llegar y poner su QR" — necesita poder verificar que el sistema es oficial.

A lo largo de toda la entrevista (al hablar de confianza, de qué lo haría reportar más seguido, y de qué cambiaría cómo se siente) repite que lo que más le importa es saber que alguien recibió y está atendiendo su reporte — necesita confirmación de recepción y seguimiento visible.

Cuando algo no vale la pena esperar, lo resuelve él mismo (compró un cable HDMI en vez de reportarlo) — necesita que reportar sea al menos tan rápido como resolverlo por su cuenta.

Decide reportar después de clase según la severidad, sin interrumpir la actividad — necesita poder reportar sin fricción en el momento, y que el sistema distinga urgencia.

No tiene visibilidad de qué pasa después de reportar — necesita transparencia del proceso interno sin tener que preguntar.

Prefiere que le avisen del progreso, no solo cuando esté terminado — necesita actualizaciones incrementales, no un estado binario resuelto/no resuelto.

Cree que reportar cambia algo si "suficientes maestros" lo hacen — percibe valor en que los reportes se agreguen o sumen peso colectivo.

Ha sentido frustración y desconfianza cuando reporta y nunca sabe qué pasó — el silencio del sistema le genera más desconfianza que la lentitud en sí.

Divergencia

Reducir la fricción entre notar y reportar — acceso directo sin buscar el QR físico; reporte de un toque para incidencias recurrentes; recordatorio pasivo sin que el docente tenga que acordarse.

Simplicidad de la interfaz — formulario con plantillas prellenadas por categoría común; reportar con foto + una frase, sin campos obligatorios extensos.

Verificar que el QR es oficial — sello institucional reconocible; QR dinámico ligado a un registro institucional; confirmación en pantalla al escanear.

Confirmación de recepción y seguimiento — correo automático al recibir el reporte; ticket con número consultable; barra de progreso con 3 estados; aviso cuando alguien del área técnica lo abre, aunque no lo resuelva aún.

Que reportar no sea más lento que resolverlo uno mismo — tiempo estimado de respuesta visible antes de reportar; reporte de un clic para incidencias menores repetidas; distinción entre "repórtalo y espera" vs. "esto resuélvelo tú si es urgente".

Reportar sin interrumpir, con urgencia marcada — selector simple de urgencia (leve/moderado/urgente); reporte por voz o foto rápida durante la clase; cola de "reportar después" para completar pendientes al salir.

Transparencia del proceso interno — panel por docente con el estado de sus reportes; historial visible de reportes anteriores; indicador de cuántos reportes están en cola antes del suyo.

Actualizaciones incrementales, no binario — estados intermedios explícitos (en revisión / esperando / programado); comentario corto del técnico aunque no esté resuelto; aviso periódico aunque no haya cambios.

Peso colectivo de los reportes — contador visible de cuántas personas reportaron lo mismo; agregación automática de duplicados en un solo caso; "reportes similares" sugeridos al crear uno nuevo.

Evitar el silencio como fuente de desconfianza — garantía de primera respuesta en un plazo fijo; cierre explícito del ciclo (aviso de resolución, no solo desaparecer); opción de reabrir o preguntar sobre un reporte sin respuesta.

Cross-pollination

Cruces internos: el sello institucional + el reporte de un toque → un QR que ya reconoce el problema más común de esa ubicación y lo reporta con un solo toque. Los estados intermedios + reportes similares → un estado de "en revisión — N personas más reportaron esto", que une transparencia con peso colectivo.

Paralelos con otros dominios, mismo problema (confianza en que "alguien lo está atendiendo"):

- **Rastreo de paquetería** — estados incrementales con notificación en cada cambio, sin que el usuario pregunte. Aplicado: recibido / asignado / en proceso / resuelto.
- **Tickets de soporte técnico** — número de ticket + plazo de primera respuesta visible (SLA). Aplicado: prometer un plazo de primera respuesta, no solo la resolución final.
- **Apps de delivery** — progreso visual, no texto. Aplicado: barra o mapa de avance en vez de lista de estados.
- **Reportes ciudadanos de infraestructura urbana** — muestran cuántos vecinos reportaron lo mismo, lo que sube la prioridad visible. Aplicado directo a la necesidad de peso colectivo que mencionó Jonatan.

Convergencia

1. **Confirmación de recepción + estados incrementales visibles** — la necesidad que Jonatan repitió más veces y de más formas distintas (preguntas 4, 9, 12, 13, 16, 17, 18). Prioridad sobre cualquier otra dirección.
2. **Reporte de baja fricción para incidencias menores** — justificado por el contraste entre "nota problemas seguido" y "reporta poco": el costo de reportar hoy no se siente proporcional al problema.
3. **Selector de urgencia sin interrumpir la actividad** — Jonatan ya usa un criterio real (severidad) para decidir cuándo reportar; el sistema debería capturarlo, no ignorarlo.
4. **Indicador de reportes similares/duplicados** — refuerza directamente su creencia de que reportar "sí cambia algo" cuando varios lo hacen.

Fase 1 — Estructura: Los 3 Engranajes

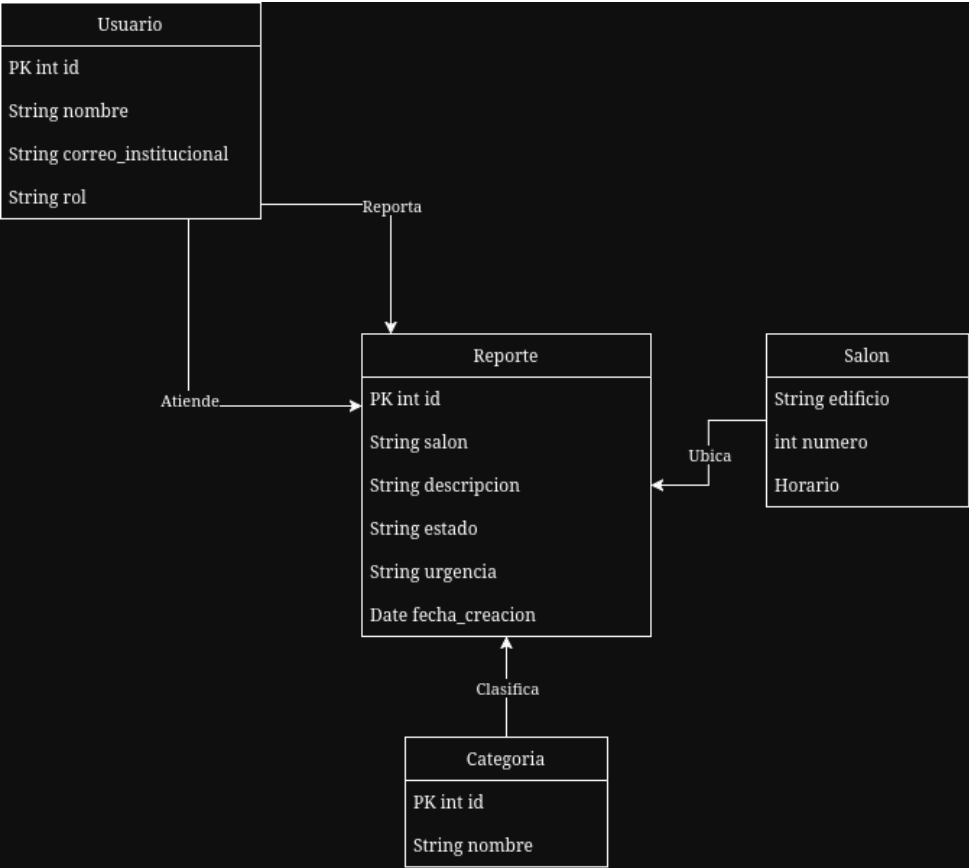
Mapa de AI (Sitemap)

[Diagrama jerárquico (árbol) de todas las pantallas de la versión definida en la Convergencia de la Fase 0 — no solo el flujo principal.]



*Categorías → ER

[Diagrama entidad-relación real: tablas con PK, FK y los campos mínimos que soportan el sitemap y los task flows de la Fase 2.]



Navegación → Rutas y Queries

Reportar

Ruta	Pantalla	Consulta
/login	Inicio de sesión	SELECT * FROM usuario WHERE correo_institucional = ?
/reportes/nuevo	Nuevo reporte	SELECT id, nombre FROM categoria y SELECT id, edificio, numero FROM salon
/reportes/nuevo/confirm	Confirmación de envío	INSERT INTO reporte (...) + INSERT INTO seguimiento (reporte_id, estado='recibido', fecha)
/reportes	Mis reportes	SELECT r.*, c.nombre AS categoria, (SELECT estado FROM seguimiento WHERE reporte_id=r.id ORDER BY fecha DESC LIMIT 1) AS estado_actual FROM reporte r JOIN categoria c ON r.categoria_id=c.id WHERE r.usuario_id = :current_user

/reportes/:id	Detalle de reportes	SELECT * FROM reporte WHERE id=:id + SELECT * FROM seguimiento WHERE reporte_id=:id ORDER BY fecha ASC
---------------	---------------------	--

Atender

Ruta	Pantalla	Consulta
/atender/bandeja	Reportes nuevos	SELECT * FROM reporte WHERE tecnico_id IS NULL ORDER BY fecha_creacion ASC
/atender/reportes/:id	Detalle + diagnostico	SELECT r.*, s.edificio, s.numero FROM reporte r JOIN salon s ON r.salon_id=s.id WHERE r.id=:id
/atender/reportes/:id/asignar	Asignarme reporte	UPDATE reporte SET tecnico_id=:current_user WHERE id=:id + INSERT INTO seguimiento (... , estado='en progreso')
/atender/mis-asignados	Mis asignados	SELECT * FROM reporte WHERE tecnico_id=:current_user ORDER BY estado, fecha_creacion

Administración

Ruta	Pantalla	Consulta
/admin/dashboard	Vista general	SELECT categoria_id, estado, COUNT(*) FROM reporte GROUP BY categoria_id, estado
/admin/pendientes	Sin actualizar	SELECT * FROM reporte WHERE estado != 'resuelto' AND fecha_creacion < NOW() - INTERVAL '15 days'
/admin/encargados	Gestion de encargados	SELECT * FROM usuario WHERE rol = 'tecnico'
/admin/categorias	Categorías y prioridades	SELECT * FROM categoria

Compartido

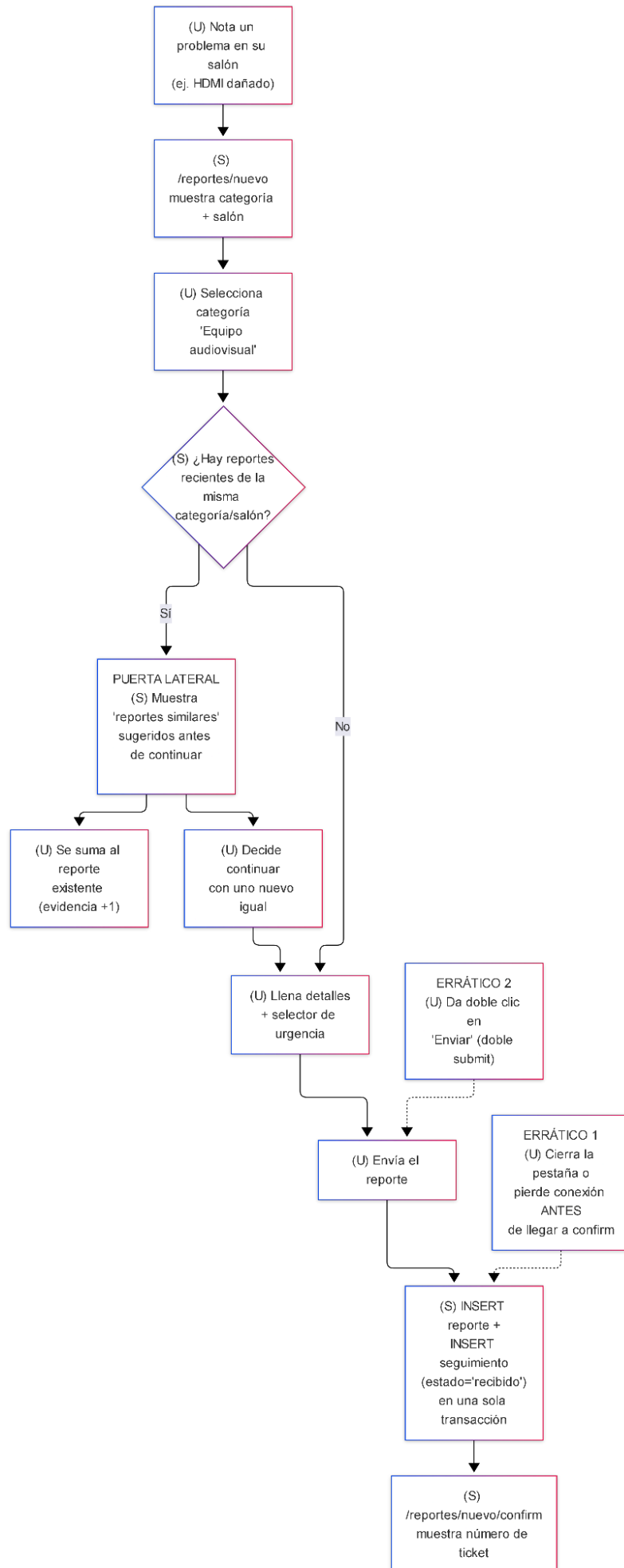
Ruta	Pantalla	Consulta
/perfil	Mi perfil	SELECT nombre, correo_institucional, rol FROM usuario WHERE id=:current_user

Fase 2 — Task Flows

Leyenda: **(U)** = acción de la persona usuaria · **(S)** = respuesta del sistema.
Cada flujo referencia las rutas de "Navegación → Rutas y Queries" (Fase 1) y las tablas del ERD (Usuario, Reporte, Categoria, Salon, Seguimiento).

Task Flow 1 — Reportar una incidencia

Necesidad de la Fase 0 que justifica este flujo: #1 de la Convergencia — *"Confirmación de recepción + estados incrementales visibles"* (la que Jonatan repitió más veces) — y #4 — *"Indicador de reportes similares/duplicados"*.



Puerta lateral: al detectar coincidencia de categoría + salón reciente, el sistema ofrece unirse a un reporte ya existente en vez de forzar la creación de uno nuevo — resuelve directamente la necesidad #4.

Comportamientos erráticos:

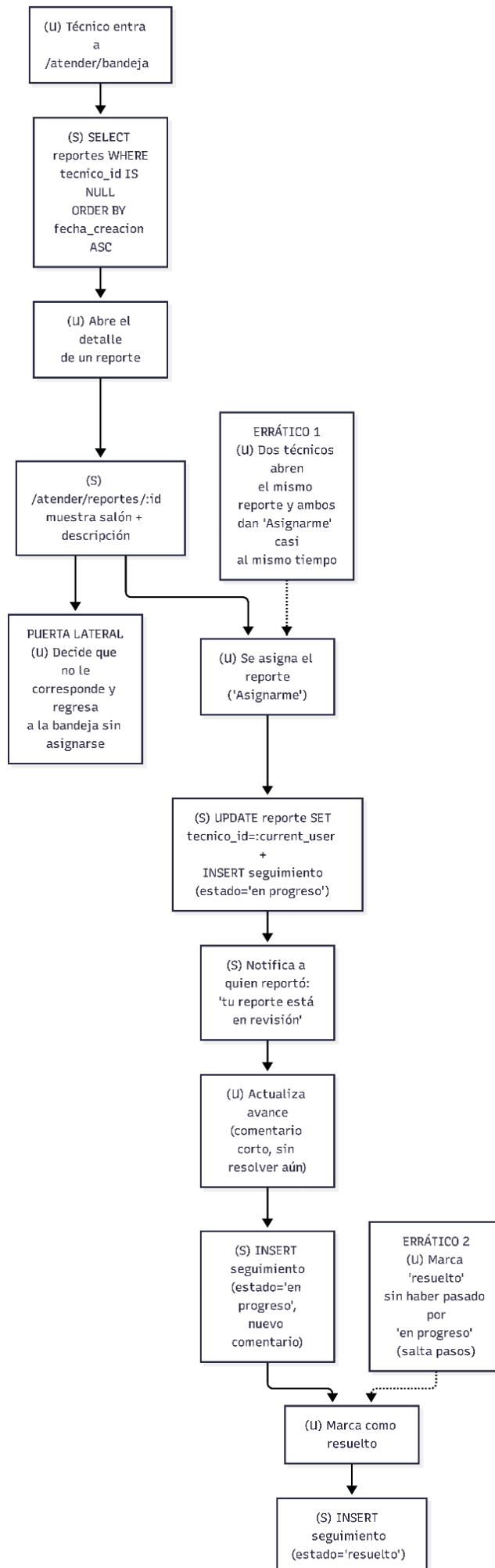
1. La persona cierra la pestaña o pierde conexión justo antes de que la transacción termine.
2. Envía el formulario dos veces por un doble clic (submit duplicado).

Respuesta de la arquitectura:

INSERT INTO reporte y el INSERT INTO seguimiento (... , estado='recibido') inicial viven en una sola transacción — si la conexión se pierde a mitad de camino, la transacción se revierte completa y no queda un Reporte huérfano sin Seguimiento (evita el estado inconsistente del errático 1). Para el doble submit (errático 2), el botón de envío se deshabilita en cuanto se dispara handleSubmit hasta que la respuesta regrese, y el id de ticket generado (uuid/autoincrement) hace que un segundo intento accidental sea idempotente a nivel de UI antes de tocar la base de datos.

Task Flow 2 — Atender y actualizar el estado de un reporte

Necesidad de la Fase 0 que justifica este flujo: #1 de la Convergencia — *"aviso cuando alguien del área técnica lo abre, aunque no lo resuelva aún" y "estados intermedios explícitos".*



Puerta lateral: desde el detalle, el técnico puede volver a la bandeja sin asignarse — no todo detalle abierto termina en asignación forzada.

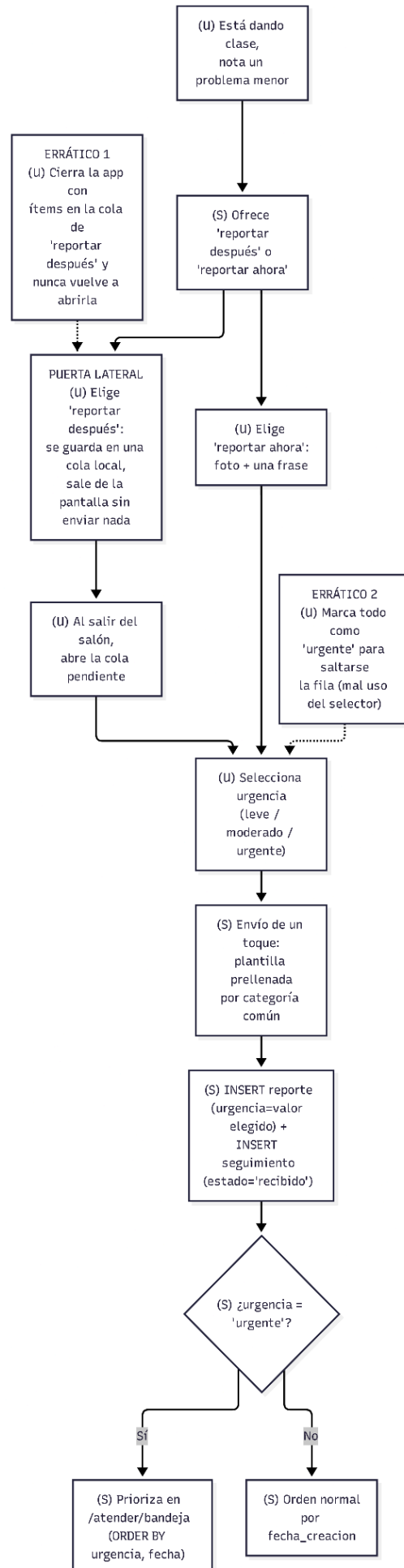
Comportamientos erráticos:

1. Dos técnicos intentan asignarse el mismo reporte casi simultáneamente (condición de carrera).
2. Un técnico marca "resuelto" directamente, sin pasar por "en progreso".

Respuesta de la arquitectura: el UPDATE reporte SET tecnico_id=:current_user solo debe ejecutarse con la condición WHERE tecnico_id IS NULL en el mismo UPDATE (no en un SELECT previo) — así, si dos técnicos disparan la asignación casi a la vez, solo el primer UPDATE afecta una fila (el segundo actualiza 0 filas) y el sistema puede avisar "ya fue tomado" sin duplicar la asignación (resuelve el errático 1). Para el errático 2, no existe una columna Reporte.estado editable directamente: el estado siempre se **agrega** como un nuevo renglón en Seguimiento (INSERT, nunca UPDATE) y la vista de historial (/reportes/:id) siempre muestra la secuencia completa — si alguien "salta" a resuelto sin pasos intermedios, el historial lo deja visible en vez de ocultarlo, en vez de impedirlo a nivel de base de datos.

Task Flow 3 — Reportar con urgencia, sin interrumpir la actividad

Necesidad de la Fase 0 que justifica este flujo: #2 — *"Reporte de baja fricción para incidencias menores"* y #3 — *"Selector de urgencia sin interrumpir la actividad"*.



Puerta lateral: "reportar después" saca a la persona del flujo sin forzar el envío inmediato — resuelve directamente que reportar no debe ser más lento que resolverlo uno mismo.

Comportamientos erráticos:

1. La cola de "reportar después" se queda llena y nunca se vacía porque la persona nunca vuelve a abrirla.
2. Uso indebido de "urgente" para saltarse el orden de atención.

Respuesta de la arquitectura: la cola de "reportar después" no vive solo en memoria del cliente — se persiste como borradores asociados a Usuario.id (o localStorage con usuario_id de respaldo), y el AppShell muestra un contador visible mientras existan borradores pendientes, en vez de depender de que la persona recuerde volver (mitiga el errático 1). Para el mal uso de "urgente" (errático 2), urgencia es una columna con dominio fijo (leve | moderado | urgente, CHECK a nivel de base de datos) — el ORDER BY en la bandeja combina urgencia con fecha_creacion como desempate, así que abusar de "urgente" solo compete dentro de su propio grupo, no le da prioridad absoluta sobre reportes anteriores igual de urgentes; el histórico de Seguimiento además queda disponible para que Supervisión detecte patrones de abuso por usuario.

Fase 3 — Entrega: Algoritmos de atención

Insumo: prototipo navegable (repositorio cima-fix-architecture-info: index.html, reportar.html, atender.html y db/schema.sql), task flows de la Fase 2 y entrevista a Lupita (estudiante de Ingeniería en Nanotecnología, 5.º semestre, jefa de grupo).

Pantallas elegidas

Inicio (index.html) — pantalla de entrada. Muestra cuatro tarjetas con el mismo peso visual (Reportar, Atender, Supervisar, Mi perfil); esta fase agrega el Johnson Box arriba de ellas.

Nuevo reporte, paso 3 (reportar.html) — pantalla de la tarea principal: es donde termina el único task flow implementado ("Reportar incidencia") y donde la persona recibe la confirmación.

Lupita confirma la necesidad #1 de la Convergencia desde otro perfil: sí levantó reportes, pero no supo qué pasó con ellos. Por eso ambas pantallas ponen el estado del reporte en la zona de mayor visibilidad.

Los números 1-5 de cada wireframe se repiten en las tres capas (superior, media y base) para poder seguir cada zona de arriba hacia abajo.

Pantalla 1 — Inicio (index.html)

Capa superior — Wireframe anotado

None

Cima Fix

1

Inicio · Reportar · Atender · Supervisar

Mi perfil

Tu último reporte: #3 · FIAD 204

2

Tecnología · EN PROGRESO

Hace 2 h

[Ver estado ▶]

[+ Reportar un problema]

3

Mis reportes abiertos: 2

1 recibido · 1 en progreso

— Secciones —

4

[Reportar] [Atender]	
[Supervisar] [Mi perfil]	
Borrador sin terminar: 1	5
[Continuar ►]	

1. Navegación global (Engranaje 2): la misma barra de todas las páginas del prototipo.
2. Johnson Box: último reporte de la persona y su estado.
3. Nivel 1: acción principal y resumen de reportes abiertos.
4. Nivel 2: las cuatro secciones del sitemap.
5. Nivel 3: borrador pendiente.

Johnson Box (2) resumen escaneable en la parte superior que responde "¿qué pasó con lo que reporté?" y permite saltar directo al estado con un clic. Sustituye la hoja de Excel que Lupita tenía que abrir para revisar el estatus. Si la persona no tiene reportes, el recuadro cambia a "Aún no tienes reportes · [Reportar un problema]".

Pirámide invertida

Nivel 1 — crítico (2,3): estado del último reporte, botón para reportar y conteo de reportes abiertos. Es lo que Lupita pidió ver sin buscar: "si fuera en tiempo real creo que estaría mejor, así uno puede visualizar de mejor manera el estado".

Nivel 2 — esencial (4): las cuatro secciones del sitemap. Siguen disponibles, pero dejan de ser lo primero que se ve; la persona que llega a Inicio casi siempre quiere reportar o revisar, no navegar por secciones.

Nivel 3 — agradable de saber (5): borrador sin terminar. Es útil, pero no bloquea nada; aprovecha el borrador que el prototipo ya guarda cuando alguien abandona el formulario.

Capa media — Task flows y estructura

Punto de partida del task flow "Reportar incidencia" — el botón de 3 lleva a reportar.html, y 5 lleva al mismo formulario con el borrador recuperado.

Punto de llegada de "Atender" — cuando un técnico presiona "Marcar 'en progreso'" en atender.html, el cambio de estado aparece en 2 la próxima vez que la persona abra Inicio.

Anterior: ninguna (pantalla de entrada). Siguiente: reportar.html (desde ③ o ⑤) o el estado del reporte (desde ②).

Capa base — Lógica y BD

Zona	Origen	Consulta
2 John son Box	Reporte, Salon, Categoria	<code>SELECT r.id, s.edificio, s.numero, c.nombre, r.estado, r.fecha_creacion FROM Reporte r JOIN Salon s ON s.id = r.salon_id JOIN Categoria c ON c.id = r.categoria_id WHERE r.usuario_id = :current_user ORDER BY r.fecha_creacion DESC LIMIT 1</code>
3 Repo rtes abier tos	Reporte	<code>SELECT estado, COUNT(*) FROM Reporte WHERE usuario_id = :current_user AND estado != 'resuelto' GROUP BY estado</code>
4 Secci ones	—	Enlaces estáticos del sitemap (sin consulta)
5 Borra dor	localStorage	Clave cima-fix:reportes:borrador (no pasa por la BD hasta que se envía)

Índices que usa: idx_reporte_fecha (orden de 2). Las consultas de 2 y 3 filtran por usuario_id, que hoy no tiene índice propio — se propone agregar idx_reporte_usuario ON Reporte(usuario_id) en schema.sql.

Pantalla 2 — Nuevo reporte, paso 3 (reportar.html)

Capa superior — Wireframe anotado

None

Cima Fix

①

Inicio · Reportar · Atender · Supervisar

Mi perfil

Nuevo reporte · Paso 3 de 3

✓ REPORTE RECIBIDO

②

Ticket #3

Tecnología · FIAD 204

Hoy 9:10 · Urgencia: moderado

— ¿Qué sigue? —

③

• Recibido ◦ En progreso ◦ Resuelto

Se avisará cuando un técnico lo tome.

Descripción: "El proyector no enciende"

— Otras acciones —

④

[Reportar otra cosa]

| [Ver todos en Atender](#) ▶ |

1 Navegación global (Engranaje 2). 2 Nivel 1 — confirmación de recepción: qué, dónde, cuándo y número de ticket. 3 Nivel 2 — qué sigue: estado actual y descripción capturada. 4 Nivel 3 — acciones secundarias.

Pirámide invertida

Nivel 1 — crítico (2) — "Reporte recibido", número de ticket, categoría (qué), salón (dónde), hora y urgencia (cuándo / qué tan grave). Responde la necesidad que Jonatan repitió más veces en la Fase 0: saber que alguien recibió el reporte.

Nivel 2 — esencial (3) — barra con los tres estados posibles (recibido, en progreso, resuelto) y la descripción enviada. Muestra desde el primer momento que el reporte va a avanzar por etapas, no en un estado binario resuelto / no resuelto (Divergencia de la Fase 0: actualizaciones incrementales). Lupita pidió exactamente eso para confiar en el sistema: "que al solucionar algún reporte, manden un correo electrónico o una notificación que ya quedó" (pregunta 9).

Nivel 3 — agradable de saber (4) — reportar otra cosa o ver todos los reportes. No cambia lo que la persona necesita saber primero.

Cuando el prototipo detecta un posible duplicado y la persona confirma que es el mismo problema, la pantalla conserva la misma estructura, pero 2 cambia a "Ya existe un reporte igual · Ticket #2" — en vez de crear otro ticket, suma el reporte al existente. Esto atiende la necesidad #4 de la Convergencia y lo que contó Lupita: los maestros "me pedían levantar más reportes para meter presión" (pregunta 5).

Capa media — Task flows y estructura

Pantalla final del task flow "Reportar incidencia" — anterior: paso 2 (Detalles + urgencia); siguiente: Inicio, donde el ticket nuevo aparece en el Johnson Box, o atender.html.

Puerta lateral — llegar por el QR del salón (reportar.html?salon=2) deja el salón preseleccionado y bloqueado. Resuelve lo que Lupita describió como lo más molesto del formulario actual: "tener que escribir el número de salón, muchas veces no sabía cuál era y tenía que salir a ver" (pregunta 3).

Errático: QR inválido (reportar.html?salon=99) — muestra el aviso "Este código no corresponde a un salón registrado" y no crea ningún ticket; nunca se llega a esta pantalla.

Errático: abandono a mitad del formulario — el borrador se guarda en cada cambio y se ofrece al volver (conecta con 5 de Inicio).

Errático: posible duplicado (misma categoría y salón hace menos de 1 hora) — se pregunta antes de crear el reporte; si es el mismo, se muestra el ticket existente.

Capa base — Lógica y BD

Zona	Origen	Consulta
2 Confirmación	Reporte, Salon, Categoría	INSERT INTO Reporte (salon_id, categoria_id, usuario_id, descripcion, urgencia) VALUES (?, ?, ?, ?, ?) — estado toma el DEFAULT 'recibido' y fecha_creacion el DEFAULT CURRENT_TIMESTAMP; después SELECT r.id, s.edificio, s.numero, c.nombre, r.urgencia, r.fecha_creacion FROM Reporte r JOIN Salon s ON s.id = r.salon_id JOIN Categoría c ON c.id = r.categoria_id WHERE r.id = :nuevo_id
3 Qué sigue	Reporte	r.estado y r.descripcion; las tres etapas de la barra salen del CHECK (estado IN ('recibido', 'en_progreso', 'resuelto'))
2 Duplicado	Reporte	SELECT id FROM Reporte WHERE salon_id = ? AND categoria_id = ? AND fecha_creacion > datetime('now', '-1 hour')
4 Acciones	—	Enlaces a reportar.html y atender.html (sin consulta)

Índices que usa: idx_reporte_salon e idx_reporte_fecha (búsqueda de duplicados); FK salon_id → Salon(id) y categoria_id → Categoria(id) garantizan que no se cree un reporte con un salón inexistente (mismo caso que el QR inválido).

Resumen: de la necesidad al elemento en pantalla

Necesidad (Fase 0 / entrevista)	Elemento	Pantalla
Ver el estado sin abrir una hoja de Excel (preguntas 4 y 8)	Johnson Box con el último reporte	Inicio 2
Demasiados pasos para revisar (pregunta 6)	Conteo de reportes abiertos en Inicio	Inicio 3
Confirmación de recepción (Convergencia #1)	"Reporte recibido" + ticket	Nuevo reporte 2
Estados incrementales y aviso al resolver (pregunta 9)	Barra recibido → en progreso → resuelto	Nuevo reporte 3
No saber el número de salón (pregunta 3)	Salón preseleccionado por QR	Nuevo reporte (puerta lateral)
Levantar reportes repetidos para presionar (pregunta 5, Convergencia #4)	Detección de duplicado	Nuevo reporte 2

Resumen: de la necesidad al elemento en pantalla

Necesidad (entrevista / Fase 0)	Elemento	Pantalla
Ver el estado sin abrir una hoja de Excel (P4, P8)	Johnson Box con último cambio	Inicio 2
Demasiados pasos para revisar (P6)	Lista de reportes activos con estado	Inicio 4
No saber el número de salón (P3)	Salón prellenado por QR	Inicio 3
Aviso cuando se resuelva (P9)	Historial de estados + correo en cada cambio	Detalle 3
Levantar reportes repetidos para presionar (P5)	Evidencia	Detalle 4

Fase 4 — Matriz Diagnóstica

Dimensión	Lógica del sistema (actual)	Lógica del usuario (con AI)
Prioridad	/atender/bandeja (Fase 1) es SELECT * FROM reporte WHERE tecnico_id IS NULL ORDER BY fecha_creacion ASC — solo ordena por antigüedad. urgencia existe en el ERD pero esa query no la usa: un "leve" de hace 5 min se atiende antes que un "urgente" de hace 2 h.	Task Flow 3 (Fase 2) ya propone la corrección pero no está reflejada en la tabla de rutas de Fase 1: ORDER BY urgencia, fecha_creacion, con urgencia como CHECK de dominio fijo. Fase 4 cierra esa inconsistencia entre lo que dice Fase 2 y lo que documentó Fase 1.

Dimensión	Lógica del sistema (actual)	Lógica del usuario (con AI)
Navegación	El sitemap general agrupa por tarea (Reportar/Atender/Supervisar), pero dentro de "Administración" vuelve a organizarse por entidad: /admin/categorias y /admin/encargados — el mismo patrón CRUD-por-tabla que el resto del sitemap evita.	Fusionar ambas pantallas en la tarea real de quien supervisa — "Ajustar cómo se priorizan y quién atiende" — en vez de dos pantallas separadas por tabla (Categoria y Usuario), consistente con el resto del menú por tarea.
Estructura de datos	/admin/dashboard (Fase 1) es <code>SELECT categoria_id, estado, COUNT(*) FROM reporte GROUP BY categoria_id, estado</code> — tabla agregada plana. A diferencia de Inicio y Nuevo reporte (rediseñadas en Fase 3 con pirámide invertida), esta pantalla nunca se tocó y sigue siendo una tabla cruda.	Aplicar el mismo criterio de Fase 3: tarjetas por categoría ordenadas por antigüedad del reporte urgente más viejo sin atender, no una tabla de conteos sin jerarquía.
Medida de éxito	El único artefacto de medición hoy es esa misma query de /admin/dashboard — cuenta volumen por categoría/estado. Es una métrica operativa (cuántos hay), no mide si el problema de Jonatan (saber qué pasó con su reporte) se resuelve.	Usar Seguimiento (siempre INSERT, nunca UPDATE) para calcular SLA de primera respuesta (tiempo recibido → primer cambio) y % de reportes que llegan a resuelto sin que la persona pregunte. La tabla ya existe en el ERD; hoy solo se usa para historial, no para medir éxito.

Fase 5 — Síntesis: Experience Stack

ENTREGA — Frontend de la atención

Johnson Box en Inicio (Pantalla 1, Nivel 1):

"Tu último reporte: #ticket · <estado> · hace <tiempo>"

Pirámide invertida en Nuevo reporte (Pantalla 2):

Nivel 1 — estado + ticket + urgencia (lo que Jonatan pidió ver primero)

Nivel 2 — barra Recibido → En progreso → Resuelto

Nivel 3 — acciones secundarias (reportar otra cosa, ver todos)

ESTRUCTURA — Middleware del movimiento

Navegación por tarea, no por tabla (sitemap Fase 1):

Reportar · Atender · Supervisar · Mi perfil

Categorías = tipo de incidencia · urgencia con dominio fijo

(leve / moderado / urgente, CHECK a nivel de BD)

Task Flows (Fase 2):

Reportar incidencia · Atender y actualizar estado ·

Reportar con urgencia sin interrumpir la actividad

CIMENTOS — Backend del problema

Design Space Exploration (Fase 0):

reportes de mantenimiento de salones — insumo: entrevista a

Jonatan Crespo Ragland (docente, uso ocasional del QR)

Necesidad oculta que gobierna todo (Convergencia #1):

confirmación de recepción + seguimiento visible —

"el silencio del sistema genera más desconfianza que la lentitud"